

Lecture 09

Introduction to Arm Cortex M4 Programming

1. Compare C language and assembly language in the context of Cortex-M4 programming.
2. Describe the typical program-generation flow (Compile → Assemble → Link → Download).
3. Explain the Compilation process using Arm-based tools.
4. Explain the different stages of the compiler in Arm-based development tools.
5. Define a program image and mention its main components in Cortex-M4.
6. Describe the structure and purpose of the vector table in a Cortex-M4 program image.
7. Explain the role of the C start-up routine in Cortex-M4 systems.
8. Illustrate the program code section of a Cortex-M4 program image.
9. Describe how the program image is stored in global memory and explain the memory map range.
10. Explain how data is stored in RAM and differentiate between static data, stack, and heap.
11. Describe how stack and heap can be defined in C using a linker file.
12. Explain how stack and heap are defined in assembly language.
13. Discuss the standard data types used in Arm programming and their sizes.
14. Explain how different data types are stored in volatile and non-volatile memory.
15. Describe how the interrupt vector table can be defined in C language.
16. Explain how peripheral registers are accessed in C using base address definitions.
17. Explain the concept of bit-band operation in Cortex-M4.
18. Compare the Read-Modify-Write operation with the bit-band operation in assembly.
19. Explain the procedure for calling a C function from assembly language.
20. Describe how parameters are passed and return values are handled when calling C functions from assembly.
21. Explain the procedure for calling an assembly function from C language.
22. Describe the register-saving requirements when calling assembly functions from C.
23. Explain embedded assembly and how assembly instructions can be written inside a C program.
24. Describe the features of the Keil μ Vision IDE for Cortex-M4 development.

Coding Examples:

1. Consider the following program:

```
int global_var = 10;
static int static_var = 20;
const int const_var = 30;

int main(void)
{
    int local_var = 40;
    static int local_static = 50;
    const int local_const = 60;

    return 0;
}
```

- i) Identify which variables are stored in FLASH (non-volatile memory).
- ii) Identify which variables are stored in SRAM (volatile memory).
- iii) Identify which variables are stored in the stack.

2. Consider the following program:

```
const char message[] = "Hello ARM";
int counter;
int buffer[50];

void main(void)
{
    int temp = 5;
}
```

Question:

Classify each variable into one of the following:

- i) Code region (FLASH)
- ii) Static data (SRAM)
- iii) Stack

3. Consider the following Cortex-M4 C program:

```
#include <stdlib.h>

const int device_id = 1001;

int system_counter = 0;

void main(void)

{

    int temp_value = 10;

    static int persistent_value = 20;

    const int local_constant = 30;

    int *dynamic_value = (int*)malloc(sizeof(int));

    *dynamic_value = 50;

}
```

- i) Identify the memory region (FLASH / SRAM Static / Stack / Heap) where each variable is stored.
- ii) Identify which variables are stored in Volatile memory and Non-volatile memory.